LAB SHEET 7

Mobility and Multi-AP Handover in Mininet-WiFi

Week	7
Duration	2 hours
Deliverable	mobility_topo.py, screenshot report (PDF), pushed to your netprog-2026 GitHub repository

Learning Objectives

By the end of this lab, you should be able to:

1. Explain the difference between random mobility models and scripted, deterministic mobility in Mininet-WiFi.

2. Configure two access points joined by a wired backbone link to form a single roaming wireless network.

3. Script deterministic station movement using net.startMobility(), net.mobility(), and net.stopMobility().

4. Monitor a moving station's association changes programmatically to observe a handover as it happens.

5. Explain what causes a handover and why overlapping AP coverage is necessary for it to succeed smoothly.

Prerequisites

• Lab 6 completed (comfort with addStation(), addAccessPoint(), configureWifiNodes(), and interpreting iw dev... link).

• Lab 5 completed (comfort with addLink() for wired backbone connections).

Software & Materials

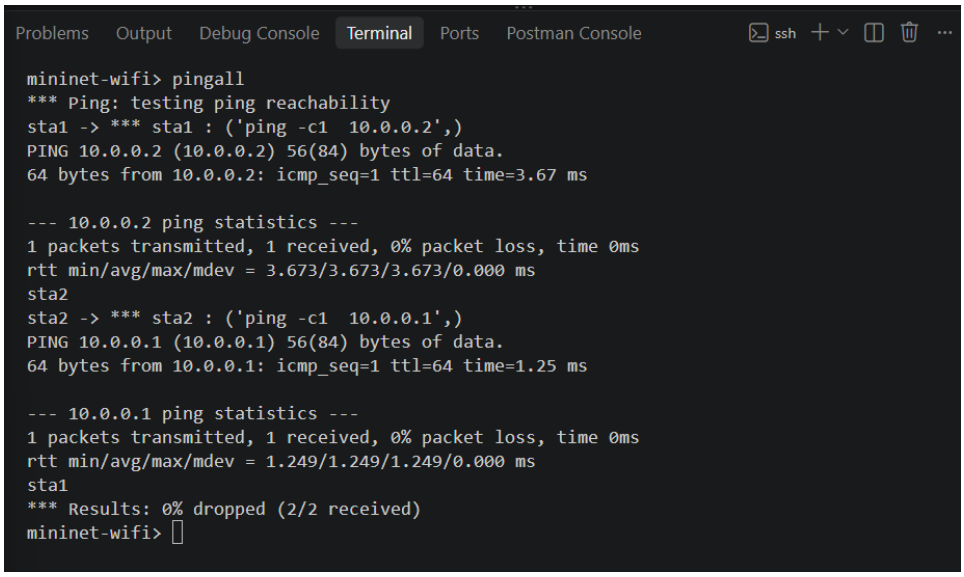
• Mininet-WiFi (installed in Lab 1), Python 3.8+
• A text editor / IDE

Lab Tasks

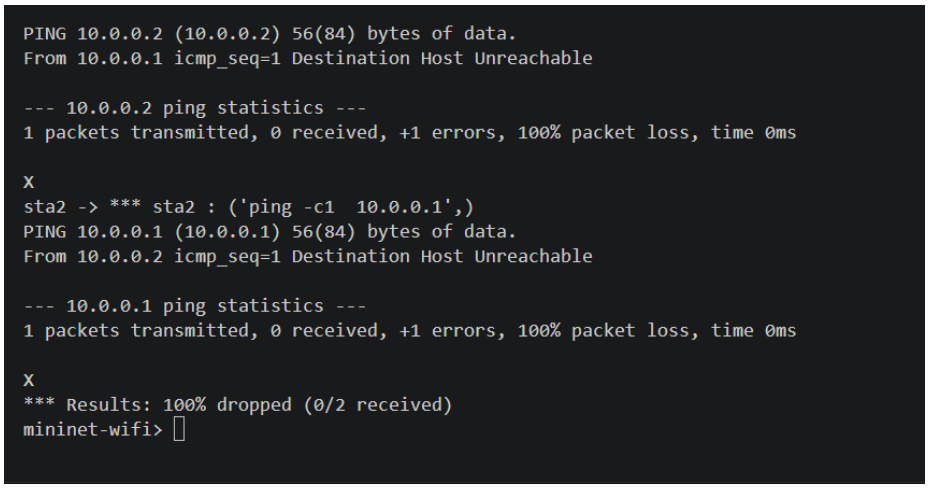
Complete the following tasks and record evidence (terminal output / screenshots) for each in your report.

Task 1: Screenshot of Part B's pingall showing sta1 (on ap1) and sta2 (on ap2) reaching each other, proving the backbone link works.

The first run :



In Mininet-WiFi, an AP is an OVS bridge that, by default, has no controller. In "secure" mode, it drops all packets; however, when switched to "standalone" mode, it functions as a standard switch, allowing for pings across APs.



I added FailMode='standalone'

Task 2: Screenshot of Part C's full per-second association log for sta3, from t=0 to t=21.

```
osboxes@osboxes:~/netprog-2026/lab07_mobility$ sudo python3 mobility_topo.py
*** Creating nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 35.00m for requested 35.00m (delta +0
.00m)
*** ap2-wlan1: minimum tx power (1 dBm) yields 35.00m for requested 35.00m (delta +0
.00m)
*** Starting mobility
*** Starting controller(s)
*** Starting L2 nodes
ap1 ap2 ...
*** Monitoring sta3 association during mobility
t=0s sta3 x=10.84m associated to: ap1-wlan1
t=1s sta3 x=14.62m associated to: ap1-wlan1
t=2s sta3 x=19.24m associated to: ap1-wlan1
t=3s sta3 x=23.86m associated to: ap1-wlan1
t=4s sta3 x=28.06m associated to: ap1-wlan1
t=5s sta3 x=32.26m associated to: ap1-wlan1
t=6s sta3 x=36.46m associated to: ap1-wlan1
t=7s sta3 x=40.66m associated to: ap1-wlan1
t=8s sta3 x=44.86m associated to: ap1-wlan1
t=9s sta3 x=49.06m associated to: ap1-wlan1
t=10s sta3 x=53.26m associated to: ap1-wlan1
t=11s sta3 x=57.46m associated to: ap2-wlan1
t=12s sta3 x=61.66m associated to: ap2-wlan1
t=13s sta3 x=65.86m associated to: ap2-wlan1
t=14s sta3 x=70.06m associated to: ap2-wlan1
t=15s sta3 x=74.26m associated to: ap2-wlan1
t=16s sta3 x=78.46m associated to: ap2-wlan1
t=17s sta3 x=83.08m associated to: ap2-wlan1
t=18s sta3 x=87.28m associated to: ap2-wlan1
t=19s sta3 x=89.80m associated to: ap2-wlan1
t=20s sta3 x=89.80m associated to: ap2-wlan1
t=21s sta3 x=89.80m associated to: ap2-wlan1
*** Starting CLI:
mininet-wifi>
```

Task 3: From your Task 2 log, report the approximate time range during which the handover occurred, and justify it using sta3's speed and the two APs' positions/ranges (Part D).

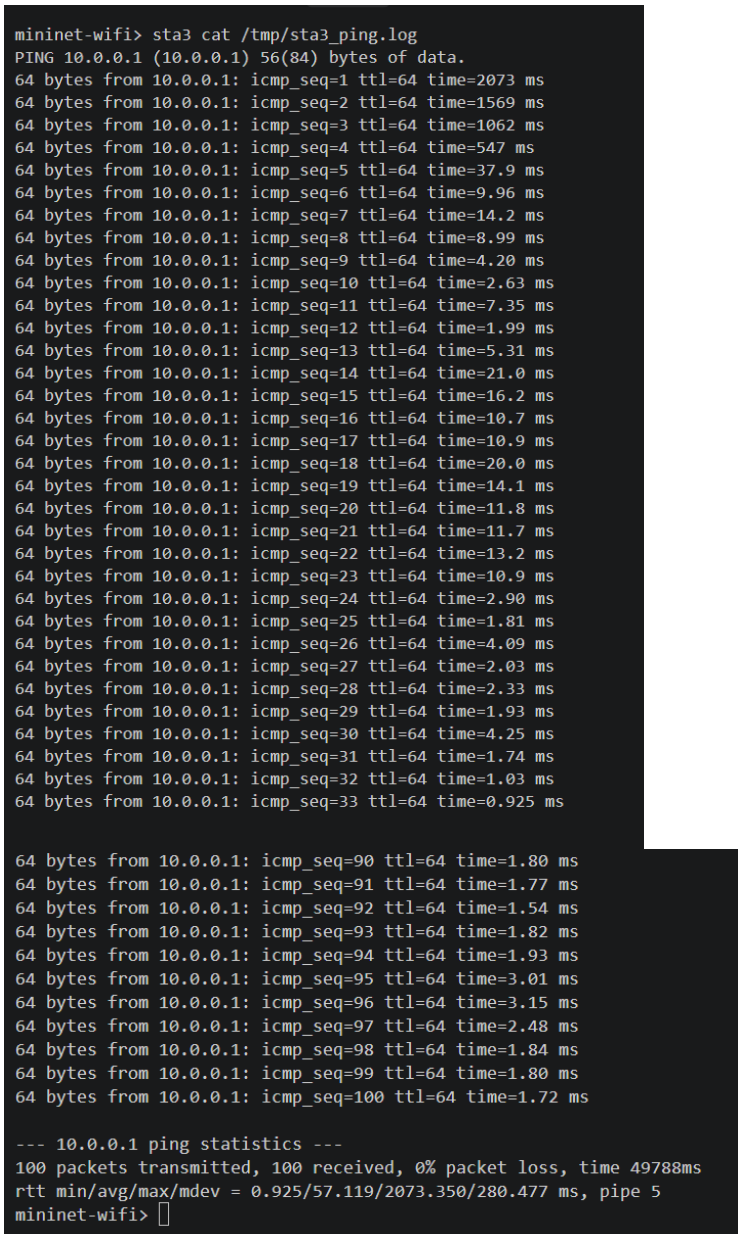
t(s)	x(m)	Associated to
9	49.06	ap1-wlan1
10	**53.26**	**ap1-wlan1**
11	**57.46**	**ap2-wlan1**

The handover happens between $t = 10$s and $t = 11$s, sta3 is located at $x = 53.26$m and is still associated with ap1-wlan1. By $t = 11$s, as it moves to $x = 57.46$m, the association changes to ap2-wlan1.

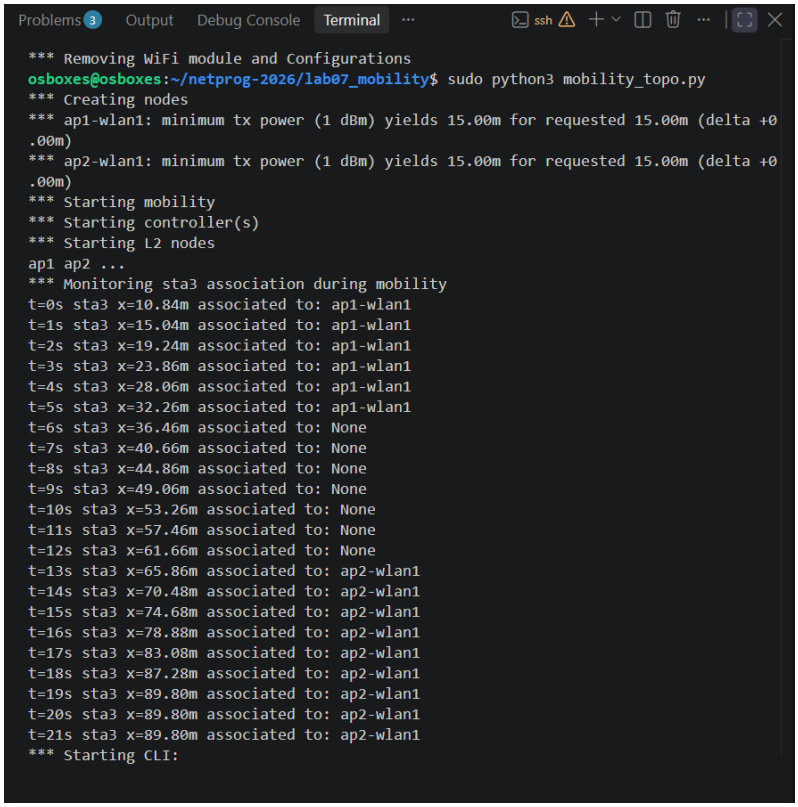
ap1 is placed at position $x = 20$m with a range of 35m, meaning its signal boundary spans from $x = 20 - 35 = -15$m to $x = 20 + 35 = \mathbf{55}$m.
ap2 is placed at position $x = 80$m with a range of 35m, meaning its signal boundary spans from $x = 80 - 35 = \mathbf{45}$m to $x = 80 + 35 = 115$m. So, the overlapping coverage zone where both AP signals are simultaneously available is between $x = 45$m and $x = 55$m

Between t =10s and t=11s, sta3 crosses the 55m mark where ap1's coverage ends, which triggers Mininet-WiFi's internal association logic to cleanly complete the handoff to ap2.

Task 4: Screenshot of Part E's ping log around the handover time. Report whether you observed any dropped pings or latency spikes and explain what you found either way.

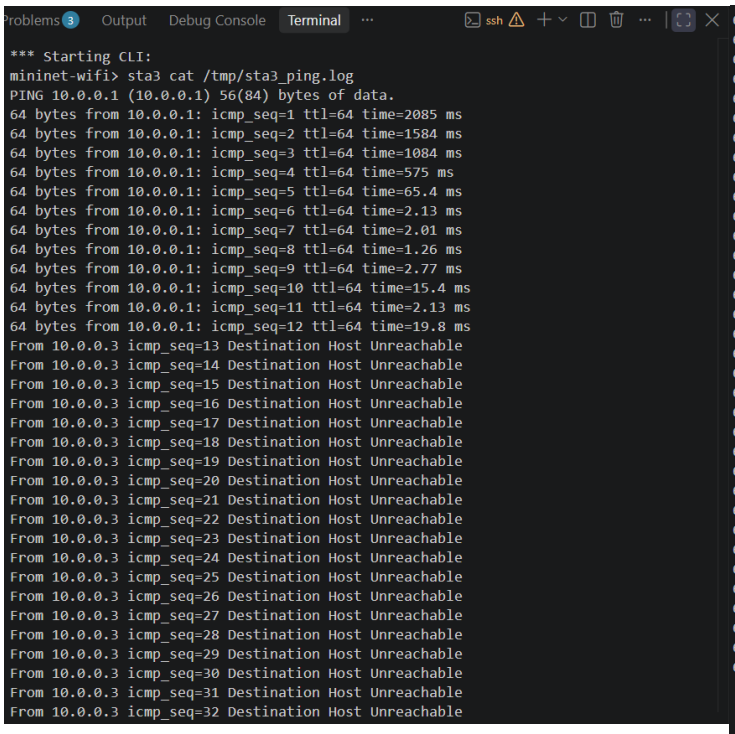


Task 5: Screenshot of Part F's association log with the reduced range=15, clearly showing a period where sta3 is associated with neither AP



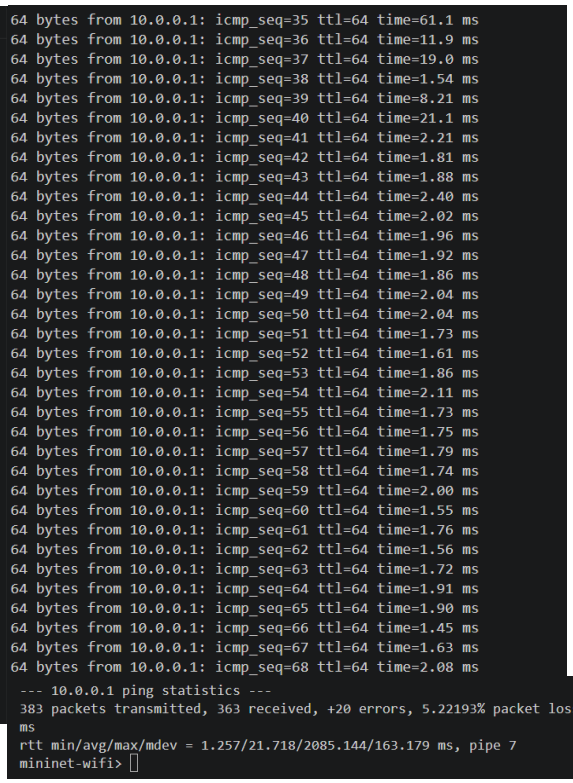
```
Problems 3  Output  Debug Console  Terminal  ...                    ssh

*** Removing WiFi module and Configurations
osboxes@osboxes:~/netprog-2026/lab07_mobility$ sudo python3 mobility_topo.py
*** Creating nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 15.00m for requested 15.00m (delta +0
.00m)
*** ap2-wlan1: minimum tx power (1 dBm) yields 15.00m for requested 15.00m (delta +0
.00m)
*** Starting mobility
*** Starting controller(s)
*** Starting L2 nodes
ap1 ap2 ...
*** Monitoring sta3 association during mobility
t=0s sta3 x=10.84m associated to: ap1-wlan1
t=1s sta3 x=15.04m associated to: ap1-wlan1
t=2s sta3 x=19.24m associated to: ap1-wlan1
t=3s sta3 x=23.86m associated to: ap1-wlan1
t=4s sta3 x=28.06m associated to: ap1-wlan1
t=5s sta3 x=32.26m associated to: ap1-wlan1
t=6s sta3 x=36.46m associated to: None
t=7s sta3 x=40.66m associated to: None
t=8s sta3 x=44.86m associated to: None
t=9s sta3 x=49.06m associated to: None
t=10s sta3 x=53.26m associated to: None
t=11s sta3 x=57.46m associated to: None
t=12s sta3 x=61.66m associated to: None
t=13s sta3 x=65.86m associated to: ap2-wlan1
t=14s sta3 x=70.48m associated to: ap2-wlan1
t=15s sta3 x=74.68m associated to: ap2-wlan1
t=16s sta3 x=78.88m associated to: ap2-wlan1
t=17s sta3 x=83.08m associated to: ap2-wlan1
t=18s sta3 x=87.28m associated to: ap2-wlan1
t=19s sta3 x=89.80m associated to: ap2-wlan1
t=20s sta3 x=89.80m associated to: ap2-wlan1
t=21s sta3 x=89.80m associated to: ap2-wlan1
*** Starting CLI:
```



```
Problems 3  Output  Debug Console  Terminal  ...                    ssh

*** Starting CLI:
mininet-wifi> sta3 cat /tmp/sta3_ping.log
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2085 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1584 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1084 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=575 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=65.4 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=2.13 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=2.01 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=1.26 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=2.77 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=15.4 ms
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=2.13 ms
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=19.8 ms
From 10.0.0.3 icmp_seq=13 Destination Host Unreachable
From 10.0.0.3 icmp_seq=14 Destination Host Unreachable
From 10.0.0.3 icmp_seq=15 Destination Host Unreachable
From 10.0.0.3 icmp_seq=16 Destination Host Unreachable
From 10.0.0.3 icmp_seq=17 Destination Host Unreachable
From 10.0.0.3 icmp_seq=18 Destination Host Unreachable
From 10.0.0.3 icmp_seq=19 Destination Host Unreachable
From 10.0.0.3 icmp_seq=20 Destination Host Unreachable
From 10.0.0.3 icmp_seq=21 Destination Host Unreachable
From 10.0.0.3 icmp_seq=22 Destination Host Unreachable
From 10.0.0.3 icmp_seq=23 Destination Host Unreachable
From 10.0.0.3 icmp_seq=24 Destination Host Unreachable
From 10.0.0.3 icmp_seq=25 Destination Host Unreachable
From 10.0.0.3 icmp_seq=26 Destination Host Unreachable
From 10.0.0.3 icmp_seq=27 Destination Host Unreachable
From 10.0.0.3 icmp_seq=28 Destination Host Unreachable
From 10.0.0.3 icmp_seq=29 Destination Host Unreachable
From 10.0.0.3 icmp_seq=30 Destination Host Unreachable
From 10.0.0.3 icmp_seq=31 Destination Host Unreachable
From 10.0.0.3 icmp_seq=32 Destination Host Unreachable
```



```
64 bytes from 10.0.0.1: icmp_seq=35 ttl=64 time=61.1 ms
64 bytes from 10.0.0.1: icmp_seq=36 ttl=64 time=11.9 ms
64 bytes from 10.0.0.1: icmp_seq=37 ttl=64 time=19.0 ms
64 bytes from 10.0.0.1: icmp_seq=38 ttl=64 time=1.54 ms
64 bytes from 10.0.0.1: icmp_seq=39 ttl=64 time=8.21 ms
64 bytes from 10.0.0.1: icmp_seq=40 ttl=64 time=21.1 ms
64 bytes from 10.0.0.1: icmp_seq=41 ttl=64 time=2.21 ms
64 bytes from 10.0.0.1: icmp_seq=42 ttl=64 time=1.81 ms
64 bytes from 10.0.0.1: icmp_seq=43 ttl=64 time=1.88 ms
64 bytes from 10.0.0.1: icmp_seq=44 ttl=64 time=2.40 ms
64 bytes from 10.0.0.1: icmp_seq=45 ttl=64 time=2.02 ms
64 bytes from 10.0.0.1: icmp_seq=46 ttl=64 time=1.96 ms
64 bytes from 10.0.0.1: icmp_seq=47 ttl=64 time=1.92 ms
64 bytes from 10.0.0.1: icmp_seq=48 ttl=64 time=1.86 ms
64 bytes from 10.0.0.1: icmp_seq=49 ttl=64 time=2.04 ms
64 bytes from 10.0.0.1: icmp_seq=50 ttl=64 time=2.04 ms
64 bytes from 10.0.0.1: icmp_seq=51 ttl=64 time=1.73 ms
64 bytes from 10.0.0.1: icmp_seq=52 ttl=64 time=1.61 ms
64 bytes from 10.0.0.1: icmp_seq=53 ttl=64 time=1.86 ms
64 bytes from 10.0.0.1: icmp_seq=54 ttl=64 time=2.11 ms
64 bytes from 10.0.0.1: icmp_seq=55 ttl=64 time=1.73 ms
64 bytes from 10.0.0.1: icmp_seq=56 ttl=64 time=1.75 ms
64 bytes from 10.0.0.1: icmp_seq=57 ttl=64 time=1.79 ms
64 bytes from 10.0.0.1: icmp_seq=58 ttl=64 time=1.74 ms
64 bytes from 10.0.0.1: icmp_seq=59 ttl=64 time=2.00 ms
64 bytes from 10.0.0.1: icmp_seq=60 ttl=64 time=1.55 ms
64 bytes from 10.0.0.1: icmp_seq=61 ttl=64 time=1.76 ms
64 bytes from 10.0.0.1: icmp_seq=62 ttl=64 time=1.56 ms
64 bytes from 10.0.0.1: icmp_seq=63 ttl=64 time=1.72 ms
64 bytes from 10.0.0.1: icmp_seq=64 ttl=64 time=1.91 ms
64 bytes from 10.0.0.1: icmp_seq=65 ttl=64 time=1.90 ms
64 bytes from 10.0.0.1: icmp_seq=66 ttl=64 time=1.45 ms
64 bytes from 10.0.0.1: icmp_seq=67 ttl=64 time=1.63 ms
64 bytes from 10.0.0.1: icmp_seq=68 ttl=64 time=2.08 ms
--- 10.0.0.1 ping statistics ---
383 packets transmitted, 363 received, +20 errors, 5.22193% packet loss, time 19154
ms
rtt min/avg/max/mdev = 1.257/21.718/2085.144/163.179 ms, pipe 7
mininet-wifi>
```

Task 6: In 2-3 sentences, connect what you observed today to how a real mobile device roams between Wi-Fi access points (or cell towers), and explain why overlapping coverage matters based on your Part F comparison.

Real mobile devices and cell phones automatically switch to a new tower or AP when the signal gets better. Overlapping coverage is very important because it prevents dropped connections while moving. Without overlapping, devices hit a dead zone and lose connection completely. In Part F, the range has been reduced to the point where there is no overlap; consequently, sta3 is left in a position where it is not connected to any AP.

Verification Checklist

#	Checkpoint	From	✓
1	Two-AP topology (Part B) builds; pingall between sta1 and sta2 succeeds across the backbone link	Task 1 (Image 2)	☑
2	Part C's monitoring loop runs for the full ~22 seconds and prints a clear ap1 -> ap2 transition	Task 2	☑
3	Handover time range reported in Task 3 is consistent with the configured speed/positions/ranges	Task 3	☑
4	Part E's ping log captured and inspected for gaps/latency around the handover	Task 4	☑
5	Part F's reduced-range run shows a clear period of no association	Task 5 (Image1,2)	☑
6	mobility_topo.py pushed with the overlapping-range (Part C/E) configuration as the final version	Image after the reflection questions	☑

Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1. **Describe the two ways to configure station movement in Mininet-WiFi (random mobility models vs. scripted mobility), and explain why scripted mobility is preferable for a reproducible classroom exercise like this one.**

   Answer Random mobility allows Mininet-WiFi to automatically assign movement patterns to nodes—for example, using `net.setMobilityModel(model="RandomWalk", ...)` whereas scripted mobility lets you manually define the position of a station at any given time using `net.mobility()`. The advantage of the scripted approach is that the same path is followed every time the simulation is run, ensuring reproducible results and allowing for the pre-calculation of handover timings to verify the accuracy of your findings capabilities that are not possible with random mobility.

2. **Explain why the two access points needed to be linked with addLink(ap1, ap2) for this lab's handover scenario to represent one realistic roaming network, rather than two independent, disconnected networks.**

   Answer If there is no link between the two access points (APs), they function as two separate networks, and traffic cannot pass between them. Even if "sta3" were to switch its connection to the other AP, it still would not be able to communicate with "sta1." In a real-world Wi-Fi deployment, every AP is connected via cable to the same switched network, creating a single, continuous network; this allows a station to move between APs while maintaining access to all the same resources.

3. **Based on Part F's coverage-gap experiment, describe the real-world Wi-Fi or cellular network design principle it demonstrates, and explains why network planners specifically try to avoid this situation.**

Answer The principle is to ensure that the coverage areas of access points (or signal towers) overlap sufficiently to avoid "dead zones." In Part F, with a range of 15, there is a gap roughly between x=35 and x=65 where "sta3" fails to connect to any access point. Network planners aim to avoid such scenarios, as users passing through these areas would experience dropped connections, interrupted calls, or stalled downloads.

```python
import time
from mn_wifi.net import Mininet_wifi
from mn_wifi.cli import CLI
from mininet.log import setLogLevel, info

SETUP = 5

def topology():
    net = Mininet_wifi()

    info("*** Creating nodes\n")
    sta1 = net.addStation('sta1', position='15,50,0')
    sta2 = net.addStation('sta2', position='85,50,0')
    sta3 = net.addStation('sta3', position='10,50,0')
    ap1 = net.addAccessPoint('ap1', ssid='netprog-ap1', mode='g', channel='1',
                             position='20,50,0', range=15, failMode='standalone')
    ap2 = net.addAccessPoint('ap2', ssid='netprog-ap2', mode='g', channel='6',
                             position='80,50,0', range=15, failMode='standalone')

    net.configureWifiNodes()
    net.addLink(ap1, ap2)

    info("*** Starting mobility\n")
    net.startMobility(time=0)
    net.mobility(sta3, 'start', time=1 + SETUP, position='10,50,0')
    net.mobility(sta3, 'stop', time=20 + SETUP, position='90,50,0')
    net.stopMobility(time=21 + SETUP)

    net.build()
    net.start()
    ap1.cmd('ovs-vsctl set-fail-mode ap1 standalone')
    ap2.cmd('ovs-vsctl set-fail-mode ap2 standalone')


    time.sleep(3)
    sta1.cmd('iw dev sta1-wlan0 connect netprog-ap1')
    sta2.cmd('iw dev sta2-wlan0 connect netprog-ap2')
    sta3.cmd('iw dev sta3-wlan0 connect netprog-ap1')
    time.sleep(2)

    sta3.cmd('ping -i 0.5 10.0.0.1 > /tmp/sta3_ping.log &')


    info("*** Monitoring sta3 association during mobility\n")
    for t in range(22):
        x = sta3.position[0]
        assoc = sta3.params.get('associatedTo') or sta3.wintfs[0].associatedTo
        print(f"t={t}s sta3 x={x:.2f}m associated to: {assoc}")
        time.sleep(1)

    CLI(net)
    net.stop()

if __name__ == "__main__":
    setLogLevel("info")
    topology()
```

The final code in Python on netprog-2026 repo in lab07_mobility folder

End of Lab Sheet 7